Open Budget Information Security Policy and Procedures

Version 1.0 · Effective Nov 2, 2026 · Owner Jacob Shade

Purpose, scope and roles

This policy sets the controls and recurring procedures Open Budget uses to identify, mitigate and monitor information security risks to consumer financial data received from Plaid.

Scope: the Open Budget application (Rust backend, React frontend, SQLite/SQLCipher database), the host it runs on, its source code repository (GitHub `jacob-shade/open-budget`), its secrets, and its vendors Plaid and Google.

Roles: Open Budget is operated by one person, Jacob Shade, who is the Security Owner. The Security Owner approves this policy, runs every procedure in it, keeps the evidence log in the last section, and is the only person with administrative access to the host, repository and vendor dashboards.

Each control below is marked **Implemented** (in place and verified) or **Planned** (committed, not yet in place). Only Implemented controls are attested to Plaid as current.

Data classification and inventory

All consumer data received from Plaid is classified **Restricted**, the highest level; there is no lower tier for Plaid data. Restricted data is encrypted at rest and in transit and is never written to logs or the repository.

Data	Class	Stored in	Protection
Plaid access tokens, item IDs	Restricted	SQLCipher database	AES-256 at rest; never sent to the browser
Accounts (name, mask, type, balances, limits)	Restricted	SQLCipher database	AES-256 at rest
Transactions (amount, date, merchant, category)	Restricted	SQLCipher database	AES-256 at rest
TOTP secrets, recovery code hashes, session token hashes	Restricted	SQLCipher database	AES-256 at rest; codes and tokens stored only as SHA-256 hashes
Database key, Plaid secret, Google client secret	Secret	`backend/.env` on the host	File mode 600; git-ignored
Google profile (email, name, picture link)	Confidential	SQLCipher database	AES-256 at rest

Data	Class	Stored in	Protection
Operational logs	Internal	Container log on the host	No balances, transactions or tokens logged
Source code	Internal	GitHub repository	No secrets or data committed

Risk management procedure

Risks are identified, rated, mitigated and re-checked on a fixed cadence, and every risk is tracked in the register below.

1. **Identify.** Add a risk to the register whenever any of these happens: a new feature or data type, a new vendor or hosting change, a dependency advisory, a Plaid or Google requirement change, an incident, or the quarterly review.
2. **Assess.** Rate likelihood and impact as Low, Medium or High. Any risk that could expose Restricted data is at least Medium impact.
3. **Mitigate.** Choose a control for every Medium or High risk, and record its status. A Low risk may be accepted, with the reason written in the register.
4. **Monitor.** Review the register **quarterly** and after every incident or significant change. Confirm each Implemented control still works, and record the review in the evidence log.

Risk register

ID	Risk	Likelihood	Impact	Control	Status
R1	Stolen or phished Google password gives access to financial data	Medium	High	Allowlist of approved Google accounts plus mandatory authenticator code	Implemented
R2	MFA code brute-forced or replayed	Low	High	5 wrong codes end the attempt; each code accepted once	Implemented
R3	Database file or a copy of it is stolen	Medium	High	Whole-database AES-256 encryption with SQLCipher	Implemented
R4	Plaid access token leaks to the browser or a log	Low	High	Tokens never leave the server, never logged, revoked when a bank is removed	Implemented
R5	Session hijacking or cross-site request forgery	Low	High	Random tokens stored only as hashes; HttpOnly, SameSite=Strict cookies; 7-day expiry	Implemented
R6	Secrets committed to the repository	Low	High	`.env` and database files git-ignored; staged files checked before each commit	Implemented

ID	Risk	Likelihood	Impact	Control	Status
R7	Database key and database sit on the same host	Medium	High	`.env` readable only by the owner (mode 600); move the key to a secrets manager	Partial
R8	Traffic between browser and app is intercepted	Low	High	App reachable only from the local machine; HTTPS with TLS 1.2+ required before any network exposure	Partial
R9	Vulnerable third-party dependency	Medium	Medium	Monthly `cargo audit` and `npm audit`; patch High and Critical findings within 7 days	Planned
R10	Host compromised by malware or lost device	Low	High	OS security updates, full-disk encryption, screen lock	Planned
R11	Database key lost, making data unrecoverable	Low	Medium	Copy of the key in a password manager, off the host	Planned
R12	Disk failure causes data loss	Low	Medium	Weekly encrypted backup stored off the host	Planned

Access control

Every request that reads or changes financial data requires a fully authenticated session; the server rejects all others with 401.

- **Application sign-in (Implemented):** Google OpenID Connect using the authorization code flow with PKCE, with state and nonce checked. Only Google accounts listed in `ALLOWED_EMAILS` are admitted.
- **Multi-factor authentication (Implemented):** a TOTP code from an authenticator app is required on every sign-in, with no option to skip. Each code is accepted only once; 5 wrong codes end the attempt. Ten single-use recovery codes are issued at enrollment and stored hashed.
- **Sessions (Implemented):** 256-bit random tokens; only their SHA-256 hash is stored. Cookies are HttpOnly and SameSite=Strict, and Secure once served over HTTPS. Sessions expire after 7 days, and a new token is issued when MFA completes.
- **Least privilege (Implemented):** only the Security Owner holds credentials for the host, the GitHub repository, the Plaid dashboard and the Google Cloud console. Each of those accounts must have MFA turned on; this is checked in the quarterly access review.
- **Access reviews (Implemented):** the allowlist, vendor dashboard users and GitHub collaborators are reviewed quarterly, and removed accounts are recorded in the evidence log.
- **Removing access:** delete the email from `ALLOWED_EMAILS`, restart the app, and delete that user's sessions.

Encryption and key management

All consumer data is encrypted at rest with AES-256 and in transit with TLS 1.2 or higher.

- **At rest (Implemented):** the entire SQLite database is encrypted with SQLCipher 4 (AES-256, per-page HMAC), including indexes and the write-ahead log. The app refuses to start if SQLCipher is unavailable, so data cannot be written unencrypted. A database created before encryption was added is converted at startup, and the plaintext original is removed only after every table's row count is verified.
- **To Plaid and Google (Implemented):** HTTPS through rustls, which supports only TLS 1.2 and 1.3.
- **Browser to app (Partial):** the app binds to the local machine only (127.0.0.1 or a loopback-only Docker port). Before it is reachable from any network, it must sit behind a TLS 1.2+ reverse proxy, with `PUBLIC_URL` set to `https://` so session cookies are marked Secure.
- **Key generation:** the database key is 256 bits from the OS random generator (`openssl rand -hex 32`). TOTP secrets, session tokens and OAuth state values also come from the OS random generator.
- **Key storage:** the key lives in `backend/.env` (mode 600, git-ignored), and a copy must be kept in a password manager off the host (R11). Moving it to a secrets manager is tracked as R7.
- **Key rotation:** rotate the database key with SQLCipher `PRAGMA rekey` if the key may have been exposed, when the host changes, and at least yearly. Rotate the Plaid and Google client secrets in their dashboards on the same triggers.

Secure development and change management

Every change goes through version control and passes the test suite before it is deployed.

1. Make changes on a branch in the GitHub repository; no code is edited directly on the host.
2. Before merging, run `cargo test` (backend, including tests for MFA codes, PKCE and ID-token validation) and `npm run build` (frontend type check). A failing check blocks the release.
3. Review the full diff before merging, giving extra scrutiny to sign-in, session, encryption and Plaid code. AI-assisted changes are reviewed the same way.
4. Before each commit, confirm no secrets or database files are staged. `.env`, `*.db` and build directories are git-ignored.
5. Deploy with `docker compose up -d --build`, then confirm the app starts, logs that it is using SQLCipher, and returns 401 for data routes without a session.
6. To roll back, redeploy the previous commit. Schema changes are forward-only migrations that run at startup.

Dependencies are pinned in `Cargo.lock` and `package-lock.json`, and builds use the locked versions (`cargo build --locked`, `npm ci`). Third-party code comes only from crates.io and npm.

Logging and monitoring

Security-relevant events are logged by the app and reviewed weekly by the Security Owner.

Event logged	Level	Reviewed
Sign-in from an account not on the allowlist (with its email)	WARN	Weekly
Google sign-in rejected (state mismatch, expired, cancelled, invalid token)	WARN	Weekly
Session ended after 5 wrong MFA codes	WARN	Weekly
Sign-in using a recovery code (with codes remaining)	WARN	Weekly
Database converted to encrypted, or failed to open (wrong key)	WARN / ERROR	Each deploy
Plaid API errors (error code and request ID) and sync failures	WARN / ERROR	Weekly
Database errors	ERROR	Weekly

Logs never contain balances, transactions, access tokens, MFA secrets or session tokens.

Weekly review: run `docker compose logs app --since 168h | grep -E "WARN|ERROR"`. Investigate anything unexpected under the incident response procedure, then record the review in the evidence log. Also check the Plaid dashboard for unexpected Items or errors.

Planned: keep logs for 90 days across redeploys (today they are lost when the container is recreated), and send an email alert on lockouts and rejected sign-ins.

Vulnerability and patch management

Known vulnerabilities are found by monthly scans and fixed within set deadlines based on severity.

- **Monthly scan:** run `cargo audit` in `backend/` and `npm audit` in `frontend/`, rebuild the Docker image to pick up base-image security updates, and record the results in the evidence log.
- **Host:** apply OS security updates at least monthly, and within 7 days for Critical advisories.
- **Advisories:** follow security notices for Plaid, Google Identity, SQLCipher and OpenSSL, and treat a relevant advisory as a trigger for an immediate scan.

Severity	Fix or mitigate within
Critical	7 days
High	7 days
Medium	30 days
Low	Next quarterly review

If a fix isn't available in time, record a compensating control, such as disabling the affected feature, as a risk in the register.

Incident response

Any suspected exposure of Restricted data is handled as an incident: stop access first, then investigate.

Triggers: an unexpected sign-in or lockout in the logs, an unknown Item in the Plaid dashboard, a leaked secret, a lost or compromised host, or a vulnerability that is being exploited.

1. **Contain (within 1 hour of detection):** stop the app (`docker compose stop`). If a secret may be exposed, rotate it: the Plaid secret, the Google client secret, or the database key (`PRAGMA rekey`). If Plaid access tokens may be exposed, remove the Items, which revokes the tokens at Plaid.
2. **Assess:** use the logs, the Plaid dashboard and Google Cloud audit logs to work out what data, which users and what time window were affected. Record findings as you go.
3. **Eradicate:** fix the root cause, such as patching, removing malware or rebuilding the host, and deploy the fix through the normal change process.
4. **Recover:** restart the app, have users sign in again, reconnect banks through Plaid Link, and watch the logs closely for 7 days.
5. **Notify:** if consumer data may have been accessed, notify Plaid as required by your Plaid agreement, and notify affected users within 72 hours of confirming the incident. Meet any state breach-notification law that applies.
6. **Learn:** within 14 days, write a short post-incident review, update the risk register and this policy, and record it in the evidence log.

Backup, retention and disposal

Consumer data is kept only while its bank connection is active, and every copy of it stays encrypted.

- **Retention (Implemented):** removing a bank in the app revokes its Plaid token and deletes its accounts and transactions immediately. Expired sessions and sign-in states are purged automatically.
- **Account deletion:** on a user's request, revoke all of their Items and delete their user record and data within 30 days.
- **Backups (Planned, R12):** copy the encrypted database file weekly to storage off the host, and keep 4 weekly copies. Backups are never decrypted outside the app. Test a restore quarterly.
- **Disposal:** before a disk or host is retired, sold or returned, securely erase or physically destroy it, after first revoking all Plaid Items that depend on it. Delete superseded backups when they age out.

Third-party and vendor management

Only two vendors handle consumer data, and both are reviewed yearly.

Vendor	What they handle	How access is controlled
Plaid	Bank connections, account and transaction data	API keys kept in `.env`; production access approved by Plaid; dashboard login limited to the Security Owner

Vendor	What they handle	How access is controlled
Google (Identity)	Sign-in: account ID, email, name, picture	OAuth client restricted to registered redirect URIs; consent screen in Testing with named test users
GitHub	Source code only, never data or secrets	Repository access limited to the Security Owner

Yearly review: confirm each vendor's security attestations (for example, SOC 2 reports), confirm data sharing still matches the privacy policy, and rotate API secrets. Any new vendor that would handle Restricted data must be reviewed and added to the risk register before use, including any hosting provider.

Operating schedule, open items and evidence

The procedures above run on this schedule, and each run is recorded in the evidence log. The next quarterly review is due Feb 2, 2027.

Procedure	How often	Section
Log review	Weekly	Logging and monitoring
Encrypted backup	Weekly (once R12 is in place)	Backup
Dependency scan, image rebuild, host updates	Monthly	Vulnerability management
Risk register and access review	Quarterly	Risk management, Access control
Backup restore test	Quarterly	Backup
Vendor review, secret rotation, policy review	Yearly	Vendors, Encryption
Post-incident review	After each incident	Incident response

Open items before attesting to Plaid

- [ ] Store a copy of `DATABASE_KEY` in a password manager off this host (R11)
- [ ] Confirm full-disk encryption, automatic security updates and screen lock on the host (R10)
- [ ] Confirm MFA on the GitHub, Plaid and Google Cloud accounts
- [ ] Put HTTPS (TLS 1.2+) in front of the app before any network exposure (R8)
- [ ] Run the first `cargo audit` and `npm audit` (R9)
- [ ] Set up weekly encrypted off-host backups and run a test restore (R12)
- [ ] Keep logs for 90 days across redeploys
- [ ] Move the database key to a secrets manager (R7)

Evidence log

Date	Activity	Result
2026-11-02	Database encrypted with SQLCipher 4.5.7	All 8 tables verified by row count; no plaintext merchant names, tokens or emails found in the database files
2026-11-02	Security tests run	10 unit tests and 28 end-to-end sign-in checks passed
2026-11-02	Google sign-in with mandatory MFA enabled	Data routes return 401 without a fully verified session
2026-11-02	Secrets file permissions set	`backend/.env` mode 600; confirmed git-ignored and absent from the repository
2026-11-02	Policy version 1.0 adopted	Initial risk register created: 12 risks